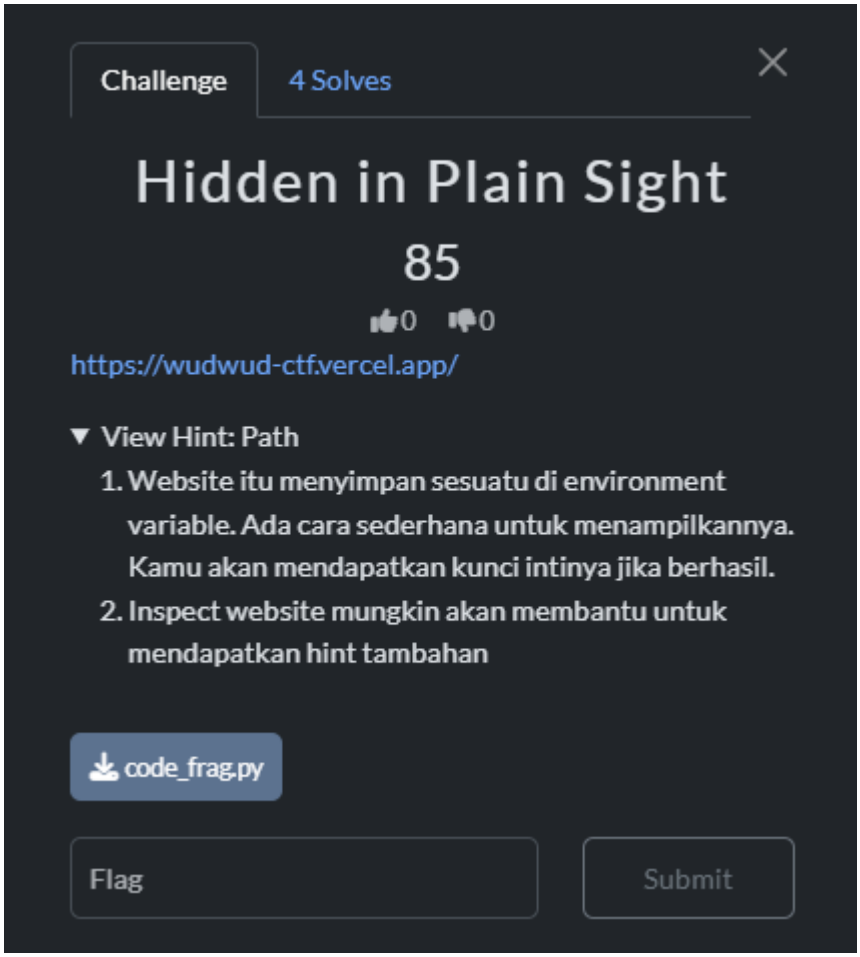
Hidden in Plain Sight

Jujur saya bingung nama tim nya apa

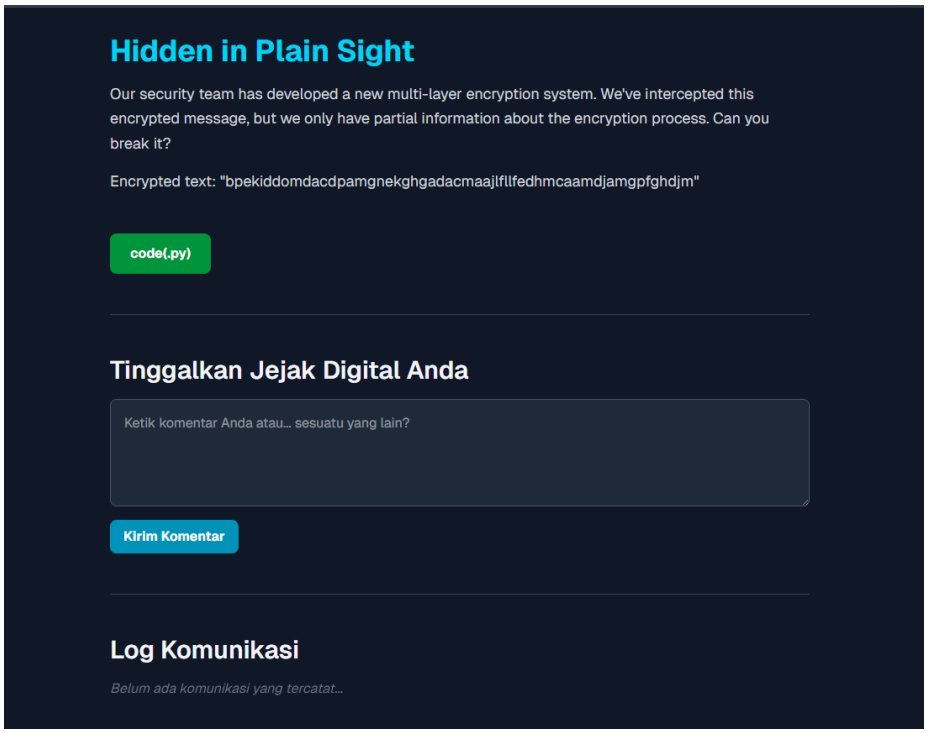
Web Exploitation

Write-up Penyelesaian

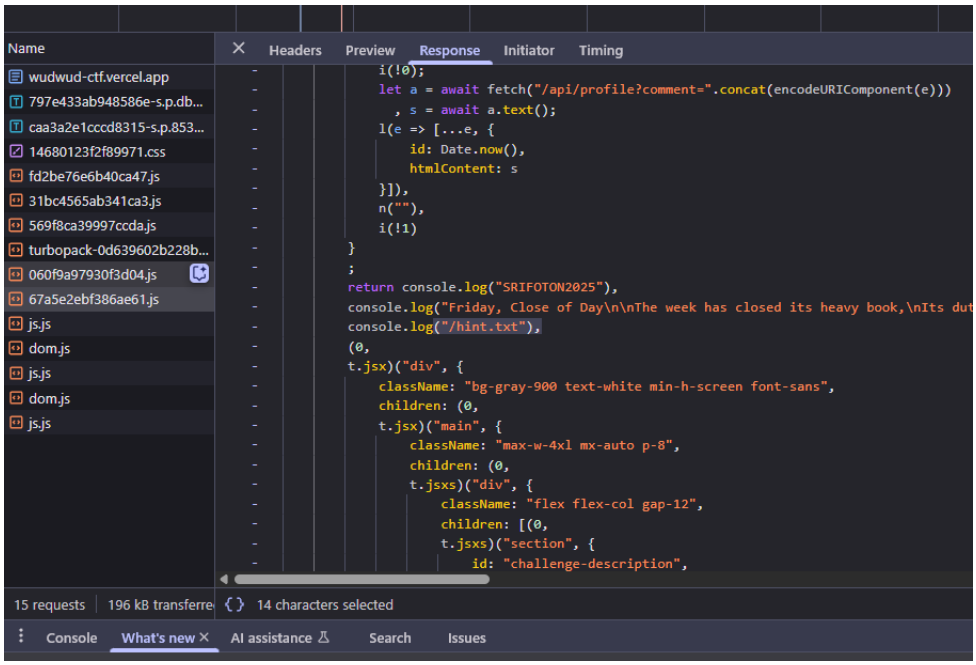
1. Dalam chall ini kita diberikan sebuah link menuju suatu website dan sebuah kode python bernama code_frag yang berisi sebuah python code buat encryption



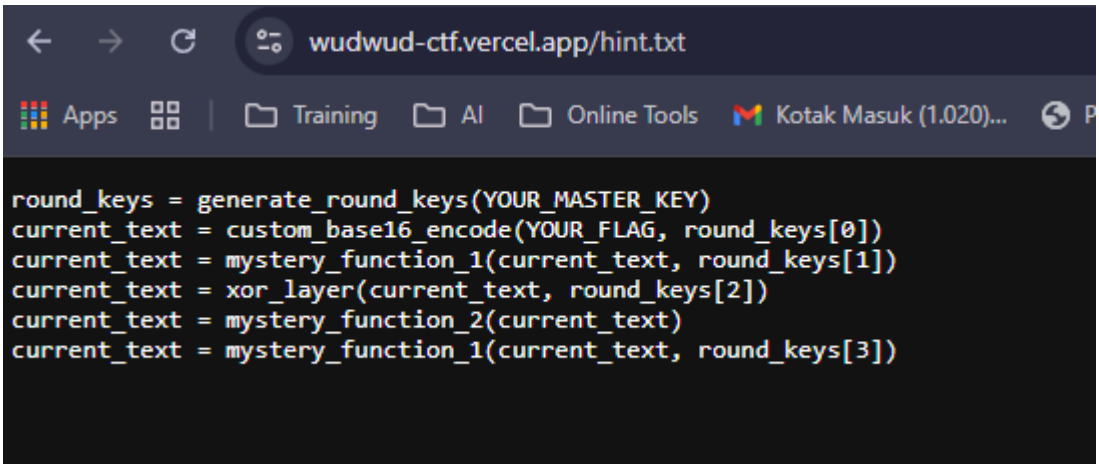
2. Dalam website ini ada sebuah tempat untuk input dan sebuah cypher text



3. Di dalam hint untuk chall dibilang bahwa ada sesuatu hint saat di inspect jadi sehabis doom scrolling di dalam inspect saya ketemu hint.txt

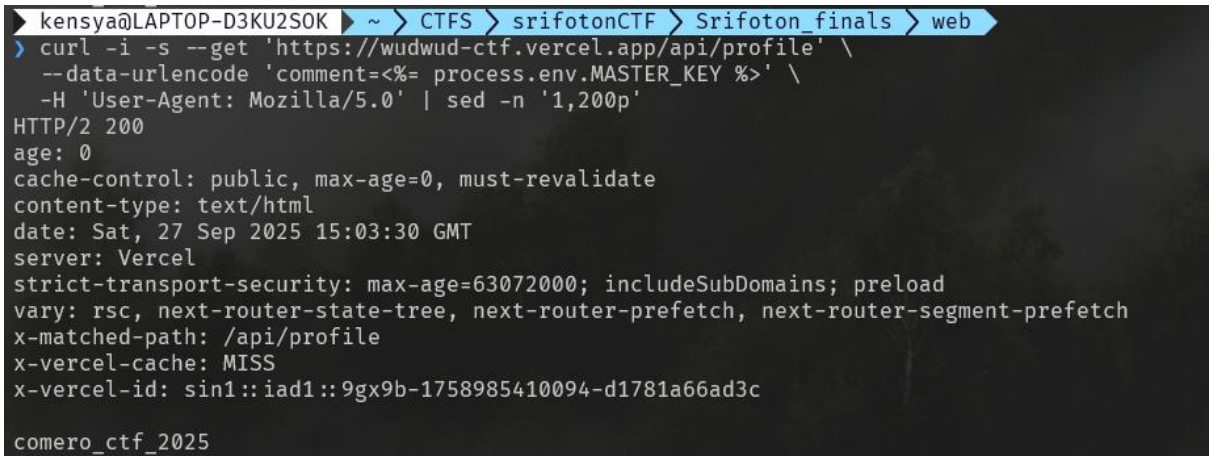


4. Selanjutnya saya menaruh hint.txt pada url dan menemukan metode atau cara encryption buat membantu saya decrypt nanti



5. Di dalam hint selanjutnya dalam chall ini dibilang bahwa ada sesuatu di environment jadi saya mencoba mencari tahu di process.env apa itu supaya bisa decrypt, saya mencoba melihat tes sederhana apakah endpoint /api/profile mem-reflect (mengembalikan) input yang kamu kirim ternyata masi ada

6. Selanjutkan saya mencoba payload template (EJS-style) yang mengakses environment variable, dan saya ketemu MASTER_KEY nya buat decryption nanti yaitu **comero_ctf_2025**



7. Sehabis ketemu MATER_KEY, cara untuk encryption/bantuan decodenya dari hint.txt, dan dari code_frag saya akan membuat script python buat decrypt cyphertext yang ada

```
//code start

#!/usr/bin/env python3

# decrypt.py

import sys, hashlib, string, itertools


ALPHABET = string.ascii_lowercase[:16]

LOWER = ord('a')


def generate_round_keys(master_key, rounds=4):

    keys = []

    for i in range(rounds):

        h = hashlib.sha256(master_key.encode() + str(i).encode()).hexdigest()

        rk = ""

        for j in range(0, 32, 2):

            idx = int(h[j:j+2], 16) % 16

            rk += ALPHABET[idx]

        keys.append(rk[:8])

    return keys


def m1_inv(text, key):

    out = []

    for i,c in enumerate(text):

        if c in ALPHABET:

            shift = ord(key[i % len(key)]) - LOWER
```

```
            old = ALPHABET.index(c)
            out.append(ALPHABET[(old - shift) % 16])
        else:
            out.append(c)
    return ''.join(out)

def m2(text, block_size=8):
    if len(text) % block_size != 0:
        text += 'a' * (block_size - len(text) % block_size)
    res = []
    for i in range(0, len(text), block_size):
        block = text[i:i+block_size]
        t = []
        for j in range(0, len(block), 2):
            if j+1 < len(block):
                t.append(block[j+1]); t.append(block[j])
            else:
                t.append(block[j])
        res.append(''.join(t))
    return ''.join(res)

def invert_xor_positions(after_xor, key):
    poss = []
    for i,c in enumerate(after_xor):
        tgt = ALPHABET.index(c)
```

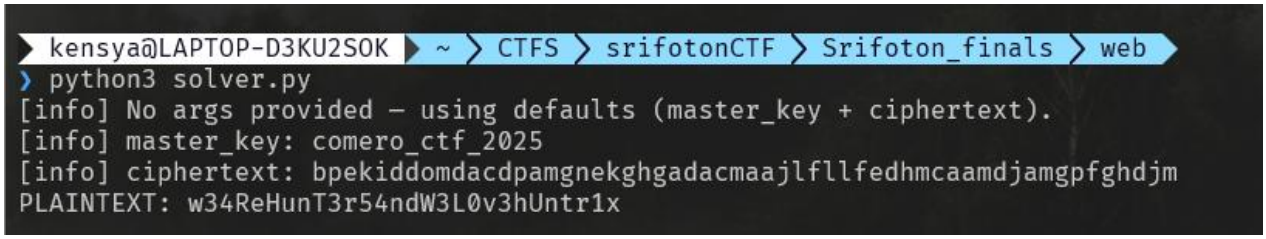
```
        kch = key[i % len(key)]
        ok = ord(kch)
        cand = []
        for ch in ALPHABET:
            if (ord(ch) ^ ok) % 16 == tgt:
                cand.append(ch)
        if not cand: return None
        poss.append(cand)
    return poss

def base16_decode_pair(a,b, key_char):
    rotation = (ord(key_char) - LOWER) % 16
    rotated = ALPHABET[rotation:] + ALPHABET[:rotation]
    if a not in rotated or b not in rotated: return None
    hi = rotated.index(a); lo = rotated.index(b)
    val = (hi << 4) | lo
    if 32 <= val <= 126:
        return chr(val)
    return None

def decrypt(master_key, ciphertext):
    rks = generate_round_keys(master_key)
    # inverse steps
    s = m1_inv(ciphertext, rks[3])
    s = m2(s)
```

```
    poss = invert_xor_positions(s, rks[2])
    if poss is None: return []
    if len(poss) % 2 != 0: return []
    byte_cands = []
    for i in range(0, len(poss), 2):
        pos0, pos1 = i, i+1
        bc = set()
        for a in poss[pos0]:
            for b in poss[pos1]:
                # reverse round_keys[1] shift
                k1 = rks[1]
                pre0 = ALPHABET[(ALPHABET.index(a) - (ord(k1[pos0 % len(k1)]) - LOWER)) % 16]
                pre1 = ALPHABET[(ALPHABET.index(b) - (ord(k1[pos1 % len(k1)]) - LOWER)) % 16]
                # decode with round_keys[0]
                k0char = rks[0][(i//2) % len(rks[0])]
                dec = base16_decode_pair(pre0, pre1, k0char)
                if dec:
                    bc.add(dec)
        if not bc: return []
        byte_cands.append(sorted(bc))
    results = []
    for prod in itertools.product(*byte_cands):
        plain = ''.join(prod)
        # quick plausibility: printable and contains letters/digits
```

```
        if all(32 <= ord(ch) <= 126 for ch in plain):
            results.append(plain)
    return results

# ----------------------
# DEFAULTS: isi master_key & ciphertext di sini
# ----------------------
DEFAULT_MASTER_KEY = "comero_ctf_2025"
DEFAULT_CIPHERTEXT = "bpekiddomdacdpamgnekghgadacmaajlfllfedhmcaamdjamgpfghdjm"
# ----------------------

if __name__ == '__main__':
    if len(sys.argv) == 3:
        mk = sys.argv[1]
        ct = sys.argv[2]
    else:
        mk = DEFAULT_MASTER_KEY
        ct = DEFAULT_CIPHERTEXT
        print(f"[info] No args provided — using defaults (master_key + ciphertext).")
        print(f"[info] master_key: {mk}")
        print(f"[info] ciphertext: {ct}")

    sols = decrypt(mk, ct)
    if not sols:
        print("No plaintext found")
```

```
    else:
        for s in sols:
            print("PLAINTEXT:", s)
//code finish
```

8. Sehabis ini saya akan run script ini, dan akhirnya mendapatkan isi flagnya



Dan ketemu flagnya
FLAG =SRIFOTONSRIFOTON{w34ReHunT3r54ndW3L0v3hUntr1x}